NAME: Praneetha Sreedharala

SRN: PES1UG24AM281

Problem Statement Repo: https://github.com/SETAPESU26/L5_42_student_grade_system

1. Objective

The objective of this lab is to improve the quality, security, readability, and maintainability of a Python program using static analysis. Pylint, Bandit, and Flake8 were used to inspect the student grade system, document issues, apply fixes, and compare results before and after the changes.

2. Program and Tools

The assigned program is a student grade system implemented in Python. It includes grade-related operations such as checking whether a student is passing, calculating GPA, generating a hash, and exporting a grade sheet.

Tool	Purpose
Pylint	Checks code quality, common programming mistakes, naming, documentation, and maintainability.
Bandit	Scans Python code for common security risks and unsafe coding patterns.
Flake8	Checks Python style and formatting, including PEP 8 rules and unused imports or variables.

3. Methodology

1. Inspected the assigned Python source file and ran the program to understand its behavior.
2. Ran Pylint, Bandit, and Flake8 against the original source and saved the initial reports.
3. Reviewed findings and documented issues and proposed fixes in an issue table.
4. Updated the source code to address code-quality, security, and style issues.
5. Ran the program and repeated all three analyses to verify the changes.
6. Saved before-and-after report files and reviewed the Git commit history.

4. Issues Identified and Fixes

The original analysis reported insecure functions and patterns, exception-handling problems, unused code, and style violations. The following table summarizes the principal issues addressed.

Issue	Category	Fix applied
Use of eval()	Security / unsafe evaluation	Removed dynamic evaluation and used direct, explicit calculation logic.
Shell command via os.system()	Security	Removed the shell invocation to avoid executing an external command unnecessarily.
MD5 hashing	Security / weak hash	Replaced MD5 with SHA-256 for a stronger hash function.
Hard-coded token	Security / hard-coded secret	Removed the hard-coded token from the source.
Random PIN generation	Security / unsuitable randomness	Used the secrets module for security-oriented random values.
Bare except / broad exception handling	Reliability and maintainability	Used more specific handling and validation rather than silently swallowing errors.
Mutable default argument	Code quality / shared state risk	Changed the default to None and initialized the value inside the function.
Unused import and variables	Code quality / style	Removed unused imports and variables.

Naming and formatting violations	Style	Improved naming and formatting to comply with tool checks.

5. Before-and-After Results

The following values were recorded from the analysis reports generated for the original and updated source files.

Tool	Before	After	Outcome
Pylint	5.51/10	10.00/10	Score improved by 4.49 points.
Bandit	6 findings (3 Low, 1 Medium, 2 High)	No issues identified	No findings in the final report.
Flake8	8 violations	No output / no violations	Style checks passed.

Verification: the updated program compiled successfully with python -m py_compile and ran from the terminal. The observed output included a passing-status result, a GPA value, a hash, a PIN, and a grade-sheet export. The final static-analysis reports showed the results summarized above.

6. Reflection

1. How useful was static analysis?

Static analysis was useful because it identified problems without requiring every issue to be discovered during manual testing. It highlighted security risks, potentially unreliable exception handling, unused code, and formatting issues. Repeating the checks after editing also provided evidence that the changes improved the code.

2. What are the differences between Pylint, Bandit, and Flake8?

Pylint focuses on general code quality, common errors, naming, and maintainability. Bandit focuses on security-sensitive patterns such as unsafe evaluation, shell execution, weak hashing, and hard-coded secrets. Flake8 focuses primarily on Python style and formatting rules, including line length and unused imports or variables. Using all three provides complementary checks.

3. What issues were found and how were they addressed?

The original reports identified eval(), os.system(), MD5 hashing, a hard-coded token, use of random for a PIN, broad exception handling, a mutable default argument, unused imports and variables, and style issues. The source was updated to remove or replace these patterns, improve exception handling, and clean up naming and formatting. The before-and-after reports were used to verify the result.

4. What was learned from the lab?

The lab demonstrated that static analysis should be part of the development workflow rather than only a final check. Different tools detect different categories of problems, so using them together is more effective than relying on one tool. It is also important to rerun the tools after changes and keep reports as evidence of verification.

7. Conclusion

The student grade system was reviewed with Pylint, Bandit, and Flake8. The updated source achieved a Pylint score of 10.00/10, Bandit reported no issues, and Flake8 reported no remaining violations. These results indicate that the reported findings were addressed according to the tools used. Static analysis improved confidence in the code's quality, security, and consistency.